

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

*I **HARI VISHNU N / 192524319** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



QUESTION 1

Given the three-address code sequence:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y.
```

Simulate the peephole optimization process. Produce the optimized instruction sequence and justify algebraic simplification, constant folding, and dead code elimination.

ANSWER:

1.1 Concept of Peephole Optimization

Peephole optimization is a local code optimization technique used by a compiler to examine a small sequence, or window, of consecutive intermediate or target instructions. The compiler searches this window for patterns that are redundant, unnecessary, or mathematically simplifiable. A replacement sequence is generated only when it produces equivalent program behavior with fewer instructions, fewer temporary variables, or simpler operations.

In this problem, the three-address code contains identity operations, a multiplication by zero, a condition that becomes known at compile time, and a statement whose usefulness must be checked after optimization. The optimization is therefore performed progressively rather than deleting instructions blindly.

1.2 Original Three-Address Code

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

1.3 Optimization Window 1 – Multiplication by One

The first instruction is $t1 = x * 1$. Multiplication by the identity value 1 does not change the operand. Therefore, $x * 1$ is equivalent to x . The multiplication instruction can be replaced by a simple assignment.

```
Before:  t1 = x * 1
After:   t1 = x
```

This is algebraic simplification. It reduces an arithmetic operation to an assignment and avoids unnecessary multiplication.

1.4 Optimization Window 2 – Addition/Subtraction Identity

The second instruction uses the value $t1$ in $t2 = t1 + y$. It cannot be simplified because y is a variable and no constant identity applies to the entire expression.

The third instruction, however, is $t3 = t2 - 0$. Subtracting zero leaves a value unchanged. Hence the instruction can be simplified to $t3 = t2$.

Before: $t3 = t2 - 0$
After: $t3 = t2$

1.5 Intermediate Code After First Simplifications

```
t1 = x
t2 = t1 + y
t3 = t2
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

At this stage, the compiler has removed two unnecessary arithmetic operations. The next window exposes a stronger optimization because multiplication by zero produces a compile-time constant.

1.6 Constant Folding and Constant Propagation

The fourth instruction is $t4 = t3 * 0$. For every possible value of $t3$, multiplication by zero produces zero. Thus the compiler can replace the entire expression by the constant assignment $t4 = 0$.

Before: $t4 = t3 * 0$
After: $t4 = 0$

This is constant folding because the compiler evaluates a constant operation during compilation. It also enables constant propagation into the following conditional instruction.

1.7 Compile-Time Evaluation of the Conditional

```
if t4 != 0 goto L1
...

After propagation:
if 0 != 0 goto L1
```

The expression $0 \neq 0$ is always false. Therefore, the jump to L1 can never be taken. The conditional branch is unreachable for this particular path and can be eliminated. This is an example of constant-condition evaluation and control-flow simplification.

1.8 Dead Code Elimination

After removing the impossible branch, the compiler checks whether any temporary computation has become dead. $t4$ was used only by the branch, so its computation is no longer required. Likewise, $t3$ exists only to produce $t4$ and therefore becomes unnecessary. The compiler can remove these dead computations.

1.9 Copy Propagation

The assignments $t1 = x$ and $t3 = t2$ are copies. Their values can be propagated into later expressions. Therefore $t2 = t1 + y$ can be represented directly as $t2 = x + y$. The final live statement $t5 = y + y$ is independent of the removed branch.

1.10 Final Optimized Sequence

```
t2 = x + y
t5 = y + y
```

1.11 Optimization Summary

Original Pattern	Optimization	Result
$x * 1$	Algebraic simplification	x
$t2 - 0$	Algebraic simplification	$t2$
$t3 * 0$	Constant folding	0
$\text{if } 0 \neq 0$	Constant condition evaluation	Branch removed
Unused temporaries	Dead code elimination	Removed
$t1/t3$ copies	Copy propagation	Direct use of values

1.12 Conclusion

The optimized sequence performs the same useful computation with substantially fewer instructions. The process demonstrates how peephole optimization can combine algebraic identities, constant folding, constant propagation, copy propagation, and dead code elimination. The final code is shorter, requires fewer temporary locations, and avoids an arithmetic operation and an impossible branch.

QUESTION 2

Partition the following intermediate code into basic blocks and construct the corresponding control flow graph. Identify all leader statements and mention the rules used to identify them:

- (1) $x = a + b$
- (2) $y = x * c$
- (3) if $y > 50$ goto 6
- (4) $z = y - d$
- (5) goto 7
- (6) $z = y + d$
- (7) $w = z * 2$

ANSWER:

2.1 Given Intermediate Code

- (1) $x = a + b$
- (2) $y = x * c$
- (3) if $y > 50$ goto 6
- (4) $z = y - d$
- (5) goto 7
- (6) $z = y + d$
- (7) $w = z * 2$

2.2 Definition of a Basic Block

A basic block is a maximal sequence of consecutive statements having a single entry point and a single exit point. Once execution enters the first statement of a basic block, the statements within the block execute sequentially unless the last statement transfers control elsewhere.

Basic-block formation is an essential compiler-design step because many optimization and data-flow techniques operate on individual blocks and the edges between them.

2.3 Rules for Finding Leaders

- Rule 1: The first statement of the entire intermediate code is a leader.
- Rule 2: The target of every conditional or unconditional jump is a leader.
- Rule 3: The statement immediately following a conditional or unconditional jump is also a leader.

2.4 Leader Identification

Statement	Leader?	Reason
(1) $x = a + b$	Yes	It is the first statement.
(2) $y = x * c$	No	Sequential statement; no control-transfer boundary.
(3) if $y > 50$ goto 6	No	It is not a jump target or the first statement.
(4) $z = y - d$	Yes	It immediately follows conditional jump (3).
(5) goto 7	No	It is inside the block beginning at (4).

(6) $z = y + d$	Yes	It is the target of goto 6 from statement (3).
(7) $w = z * 2$	Yes	It is the target of goto 7 from statement (5).

2.5 Basic Block Construction

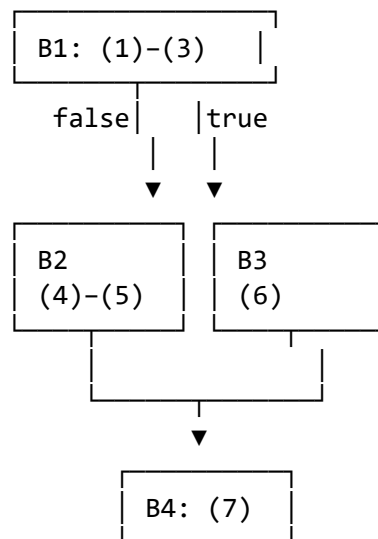
Using the leaders 1, 4, 6, and 7 as boundaries, the code is partitioned into four blocks.

Block	Statements	Contents
B1	(1)–(3)	Computes x , computes y , tests $y > 50$.
B2	(4)–(5)	Computes $z = y - d$ and jumps to 7.
B3	(6)	Computes $z = y + d$.
B4	(7)	Computes $w = z * 2$.

2.6 Internal Structure

B1 contains the condition and therefore has two possible successors. If $y > 50$ is true, execution transfers to B3. If the condition is false, execution continues to B2. B2 always transfers to B4 because statement (5) is an unconditional goto 7.

2.7 Control Flow Graph



2.8 CFG Edges

Edge	Meaning
$B1 \rightarrow B2$	False outcome of $y > 50$; normal control proceeds to statement (4).
$B1 \rightarrow B3$	True outcome of $y > 50$; control jumps to statement (6).
$B2 \rightarrow B4$	Unconditional goto 7 transfers control to statement (7).
$B3 \rightarrow B4$	After statement (6), normal sequential execution reaches statement (7).

2.9 Execution Paths

There are two principal paths through the graph. The first is $B1 \rightarrow B2 \rightarrow B4$, which occurs when y is not greater than 50. In this path, z is calculated as $y - d$. The second is $B1 \rightarrow B3 \rightarrow B4$, which occurs when y is greater than 50. In this path, z is calculated as $y + d$.

Both paths merge at $B4$. This merging point is important for subsequent data-flow analysis because the value of z reaching $B4$ depends on which predecessor block was executed.

2.10 Why the CFG Is Important

- It represents possible execution paths in a precise graph form.
- It provides the structural basis for reaching-definitions analysis.
- It helps identify loops, branches, join points, and unreachable regions.
- It allows optimizations to be applied while respecting control-flow dependencies.
- It supports later phases such as code generation and register allocation.

2.11 Final Answer

The leaders are statements 1, 4, 6, and 7. The four basic blocks are $B1$ = statements 1–3, $B2$ = statements 4–5, $B3$ = statement 6, and $B4$ = statement 7. The CFG contains the edges $B1 \rightarrow B2$, $B1 \rightarrow B3$, $B2 \rightarrow B4$, and $B3 \rightarrow B4$. This accurately captures the conditional branch and the merge of both execution paths.

QUESTION 3

Construct the **DAG** for the following basic block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

Using the DAG, identify all common sub-expressions. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

ANSWER:

3.1 Given Basic Block

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

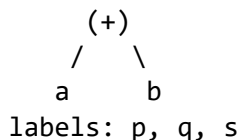
3.2 Purpose of a DAG

A Directed Acyclic Graph is an intermediate representation used for a basic block to expose common sub-expressions and redundant computations. Each leaf represents an input value, while an internal node represents an operation. If the same operation with the same operands occurs repeatedly, the DAG can represent it using one shared node.

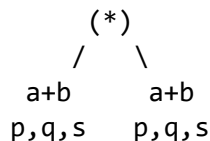
The given block is a clear example because the expression $a + b$ appears in three different assignments.

3.3 DAG Construction Process

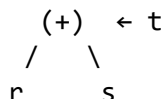
First, create leaf nodes for a and b . Then create one addition node representing $a + b$. The names p , q , and s are attached to the same node because all three assignments compute the identical value.



Next, $r = p * q$ uses the same addition result for both operands. Therefore a multiplication node is created whose two incoming edges both refer to the $a + b$ node.

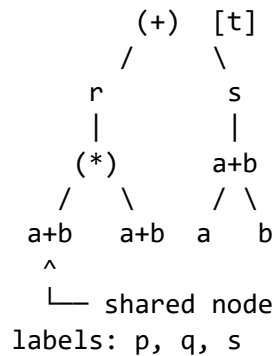


Finally, $t = r + s$ creates an addition node using r and the already available common value represented by s .



|
a+b

3.4 Complete DAG Representation



3.5 Common Sub-Expression

The expression $a + b$ is the common sub-expression. It occurs in the assignments to p , q , and s . Recomputing it three times would perform the same addition repeatedly. The DAG therefore stores one computation and reuses its result.

3.6 Optimized Intermediate Code

Introduce one temporary for the shared expression and reuse it for all dependent computations.

```

t1 = a + b
p = t1
q = t1
r = t1 * t1
s = t1
t = r + t1

```

The assignments to p , q , and s can also be treated as value aliases when the compiler's intermediate representation permits it. The important optimization is that $a + b$ is evaluated exactly once.

3.7 Simple Target Code

```

LOAD R0, a
ADD  R0, b
MOV  p, R0
MOV  q, R0

MOV  R1, R0
MUL  R1, R0

MOV  s, R0
ADD  R1, R0
MOV  t, R1

```

3.8 Register Allocation Decisions

Stage	Register	Decision
$a + b$	R0	R0 stores the shared expression because it has several future uses.
p and q	R0	The same value is copied without

		recomputing $a + b$.
$r = t1 * t1$	R1	R1 receives a copy so R0 can preserve the common value.
$s = t1$	R0	The common value remains available in R0.
$t = r + s$	R1	R1 contains r and can add the shared value in R0.

3.9 Why the DAG Improves Code

- It detects identical expressions automatically within the basic block.
- It prevents repeated evaluation of $a + b$.
- It exposes the reuse of $t1$ in both operands of r .
- It supports better register allocation by identifying values with multiple future uses.
- It reduces arithmetic instructions and can reduce memory traffic.

3.10 Final Analysis

Without DAG-based optimization, the compiler might generate three separate ADD operations for $a + b$. With the DAG, only one ADD is needed. The resulting target code therefore performs less work while preserving the values of p , q , r , s , and t . The example demonstrates the direct connection between DAG construction, common sub-expression elimination, and efficient target-code generation.

QUESTION 4

For a target machine with two registers R0 and R1, simulate the working of a simple code generator for the code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

Show the register descriptor and address descriptor after each instruction is processed.

ANSWER:

4.1 Given Three-Address Code

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

4.2 Register Descriptor

A register descriptor records which temporary or variable value is currently stored in each machine register. It prevents unnecessary LOAD instructions and helps the code generator decide which register can be reused.

4.3 Address Descriptor

An address descriptor records the locations where the current value of a variable or temporary is available. A value may be available in a register, in memory, or in both locations. The descriptor helps the compiler determine whether a STORE is required before a register is overwritten.

4.4 Initial State

Register	Contents
R0	Empty
R1	Empty

All source variables a, b, c, d, e, and f are initially assumed to be available in memory.

4.5 Generate $t1 = a + b$

```
LOAD R0, a
ADD  R0, b
```

After these instructions, R0 contains t1. R1 remains available for the next independent computation.

Descriptor	State
Register descriptor	R0 = t1; R1 = empty
Address descriptor	t1 = R0; a,b remain in memory

4.6 Generate $t2 = c * d$

```
LOAD R1, c
MUL  R1, d
```

R1 is selected because R0 is still holding t1, which will be needed to calculate t3.

Descriptor	State
Register descriptor	R0 = t1; R1 = t2
Address descriptor	t1 = R0; t2 = R1; source operands remain in memory

4.7 Generate $t3 = t1 - t2$

```
SUB R0, R1
```

The operation can use R0 as the destination because t1 is in R0 and t2 is in R1. After subtraction, R0 contains t3.

Descriptor	State
Register descriptor	R0 = t3; R1 = t2
Address descriptor	t3 = R0; t2 = R1

4.8 Generate $t4 = e + f$

The next operation needs a free register. R1 can be reused because t2 has already participated in the computation of t3. A STORE is conceptually required only if t2 must remain available later; under normal temporary lifetime analysis, t2 is dead after t3.

```
LOAD R1, e
ADD  R1, f
```

Descriptor	State
Register descriptor	R0 = t3; R1 = t4
Address descriptor	t3 = R0; t4 = R1

4.9 Generate $t5 = t3 / t4$

```
DIV R0, R1
```

R0 already contains t3 and R1 contains t4. Therefore, the final division can be performed directly without loading t3 again.

Descriptor	State
Register descriptor	R0 = t5; R1 = t4
Address descriptor	t5 = R0; t4 = R1

4.10 Complete Register Descriptor Trace

Stage	R0	R1
Initial	Empty	Empty
After t1	t1	Empty
After t2	t1	t2
After t3	t3	t2
After t4	t3	t4
After t5	t5	t4

4.11 Address Descriptor Trace

Temporary	Location after creation	Lifetime observation
t1	R0	Live until t3 is computed.
t2	R1	Live only until t3 is computed.
t3	R0	Live until t5 is computed.
t4	R1	Live until t5 is computed.
t5	R0	Final result.

4.12 Register Reuse and Optimization

- R0 is retained for t1 and then overwritten by t3 because t1 is consumed.
- R1 stores t2 and is safely reused for t4 after t2 becomes dead.
- The final operation uses the existing register values directly.
- Descriptor information reduces unnecessary memory loads and stores.

4.13 Conclusion

The simulation shows how a simple code generator uses register and address descriptors to manage only two registers efficiently. The descriptors provide the information needed to reuse registers safely, preserve live values, and avoid redundant memory operations. This is a fundamental part of target-code generation and register allocation.

QUESTION 5

For a RISC-style target machine with instructions LOAD, STORE, ADD, SUB, MUL, MOV, simulate target code generation for the expression:

$$(a - b) * (c + d) - e$$

Show the complete sequence of target instructions generated. Also explain the runtime storage management and stack frame layout used during the computation.

Given the three-address code sequence:

$t1 = a + b,$

$t2 = t1 + 0,$

$t3 = t2 * 1,$

if $t3 == 0$ goto L1,

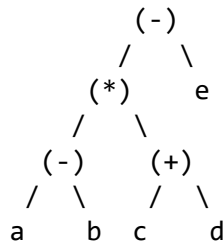
$t4 = t3 + t3$

ANSWER:

5.1 Source Expression

$$((a - b) * (c + d)) - e$$

5.2 Expression Tree



The expression tree establishes the evaluation order. First compute $a - b$, then $c + d$, multiply the two intermediate values, and finally subtract e .

5.3 Intermediate Representation

$t1 = a - b$

$t2 = c + d$

$t3 = t1 * t2$

$t4 = t3 - e$

5.4 Target Instructions

LOAD R0, a

LOAD R1, b

SUB R0, R1

STORE T1, R0

LOAD R0, c

LOAD R1, d

ADD R0, R1

STORE T2, R0

```

LOAD R0, T1
LOAD R1, T2
MUL  R0, R1

```

```

LOAD R1, e
SUB  R0, R1
STORE result, R0

```

5.5 Instruction Explanation

Instruction	Purpose
LOAD R0, a	Load first operand a.
LOAD R1, b	Load second operand b.
SUB R0, R1	Calculate $a - b$.
STORE T1, R0	Save first intermediate result.
LOAD R0, c	Load c.
LOAD R1, d	Load d.
ADD R0, R1	Calculate $c + d$.
STORE T2, R0	Save second intermediate result.
LOAD R0, T1	Reload $a - b$.
LOAD R1, T2	Reload $c + d$.
MUL R0, R1	Calculate the product.
LOAD R1, e	Load e.
SUB R0, R1	Subtract e.
STORE result, R0	Save final result.

5.6 Runtime Storage Management

Runtime storage management deals with the allocation and lifetime of memory during program execution. A compiler must decide where variables, temporaries, parameters, return information, and saved machine state are stored. For a procedure-based program, the run-time stack is commonly used to hold an activation record for each active procedure call.

The temporary values T1 and T2 in this simulation have short lifetimes. They may be stored in the current stack frame when registers are required for another sub-expression. Registers are used for active arithmetic operations because register access is faster than memory access.

5.7 Conceptual Stack Frame

Higher Memory Addresses

Return address
Saved registers / frame data
Parameters / arguments
Local variables

Temporary T1
Temporary T2
Alignment / reserved space

Lower Memory Addresses

This is a conceptual stack-frame layout. The exact offsets, register-saving rules, stack growth direction, and calling convention are architecture-dependent.

5.8 Storage Lifetime During Evaluation

Value	When Created	When Needed	Storage Consideration
$T1 = a - b$	After first SUB	During multiplication	Can be stored in stack temporary or memory.
$T2 = c + d$	After ADD	During multiplication	Can be stored while registers are reused.
$T3 = T1 * T2$	After MUL	Final subtraction	Kept in R0 for immediate use.
$T4 = T3 - e$	After final SUB	Program result	Stored in result location.

5.9 Register and Memory Interaction

The two-register limitation makes temporary storage important. R0 and R1 are sufficient for one binary arithmetic operation, but they cannot simultaneously retain every intermediate value and all new operands. STORE instructions therefore preserve T1 and T2 while registers are reused. This is a standard code-generation trade-off between register pressure and memory traffic.

5.10 Possible Optimization

If the target architecture and register allocator provide enough registers, the compiler could retain T1 in one register while computing $c + d$ in another, eliminating some STORE and LOAD operations. With only R0 and R1 and the instruction set specified in the question, the shown sequence is a safe and clear implementation.

5.11 Final Conclusion

The generated target code correctly evaluates $(a - b) \times (c + d) - e$ using the specified RISC-style instructions. The solution demonstrates instruction selection, temporary management, register reuse, memory storage, and activation-record organization. Together, these steps illustrate how a compiler transforms an intermediate expression into executable target-machine operations while managing limited hardware resources.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation